



Санкт-Петербургский государственный университет
Кафедра системного программирования

Экспериментальное исследование алгоритмов для регулярных запросов

Максим Александрович Родионов, группа 23.Б15-мм

Санкт-Петербург
2025

Постановка задачи

В данной работе исследуется производительность решения задачи достижимости с регулярными ограничениями¹ на ориентированных графах двух алгоритмов на основе достижимости:

- между всеми парами вершин (`tensor_based_rpq`)
- для заданного множества стартовых вершин (`ms_bfs_based_rpq`)

Исследование направлено на ответы на следующие вопросы:

- Какое представление разреженных матриц и векторов лучше подходит для каждой из решаемых задач?
- Начиная с какого размера стартового множества выгоднее решать задачу для всех пар и выбирать нужные?

¹Regular Path Querying, RPQ

Описание эксперимента

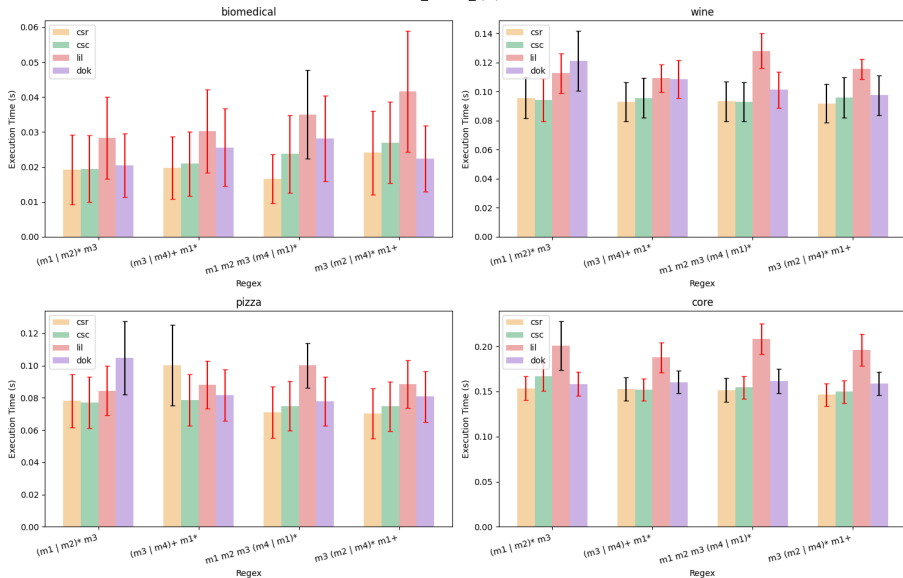
- Исследуемые решения:
 - ▶ `tensor_based_rpq`
 - ▶ `ms_bfs_based_rpq`
- Графы:
 - ▶ biomedical (341 вершин, 459 рёбер)
 - ▶ wine (733 вершин, 1839 рёбер)
 - ▶ pizza (671 вершин, 1980 рёбер)
 - ▶ core (1323 вершин, 2752 рёбер)
- Запросы:
 - ▶ $(m1 \mid m2) * m3$
 - ▶ $(m3 \mid m4) + m1 *$
 - ▶ $m1 \ m2 \ m3 \ (m4 \mid m1) *$
 - ▶ $m3 \ (m2 \mid m4) * m1 +$
- Количество стартовых вершин:
 - ▶ 5%, 10%, 20%, 30%, 45%, 60%, 80%, 100%
- форматы разреженных матриц:
 - ▶ `csr_array`, `csc_array`, `dok_array`, `lil_array`

Тестовый стенд

- Процессор: Intel Core i7-11370H с зафиксированной частотой 3.3 GHz
(**sudo cpupower frequency-set -min 3300MHz -max 3300MHz**)
 - ▶ Кэш L1: 80 КБ на ядро
 - ▶ Кэш L2: 1.25 МБ на ядро
 - ▶ Общий кэш L3: 12 МБ
- ОЗУ: 16ГБ
- ОС: Linux Kubuntu 24.04
- Версия Python: 3.12.5
- Настройки производительности: Использовался режим performance (**sudo cpupower frequency-set -g performance**)

Результаты по вопросу №1 для tensor_based_rpq

Performance of tensor_based_rpq (Start Vertices: 50%)

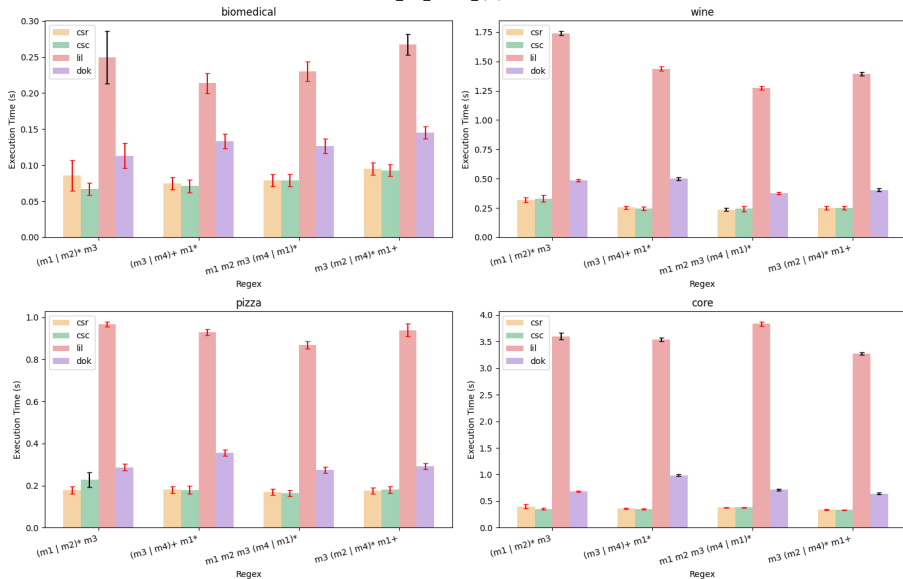


Вывод для tensor_based_rpq

- Сравнение производительности:
 - ▶ **CSR** – наиболее быстрый формат
 - ▶ **CSC** – практически идентичен **CSR** по времени, иногда чуть медленнее
 - ▶ **LIL** – медленнее всех остальных
 - ▶ **DOK** – также медленнее, чем **CSR/CSC**, но быстрее **LIL**
- Обоснование:
 - ▶ Алгоритм включает построение тензорного произведения и возведение матрицы в степень, что требует эффективного умножения и сложения разреженных матриц, для чего меньше подходит **LIL** и **DOK**, при этом **CSR** обеспечивает более быстрый доступ по строкам, что необходимо при построении результата.

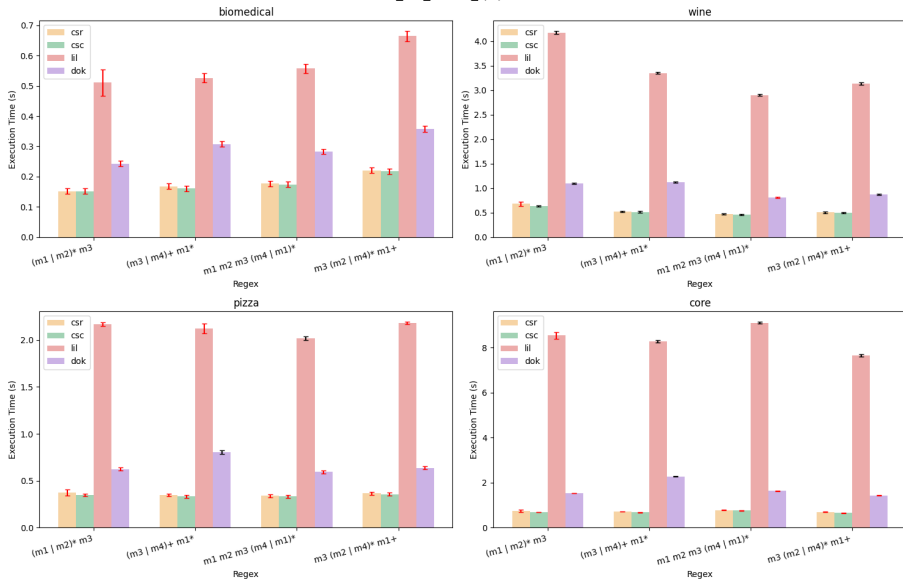
Результаты по вопросу №1 для ms_bfs_based_rpq (1/2)

Performance of ms_bfs_based_rpq (Start Vertices: 10%)



Результаты по вопросу №1 для ms_bfs_based_rpq (2/2)

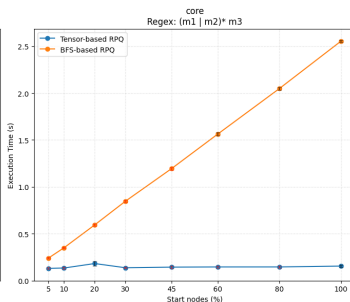
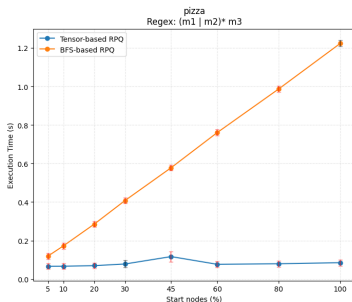
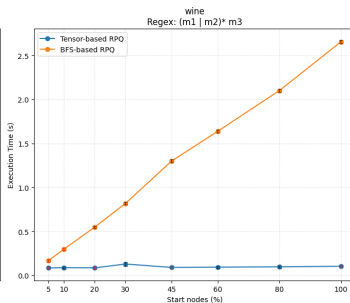
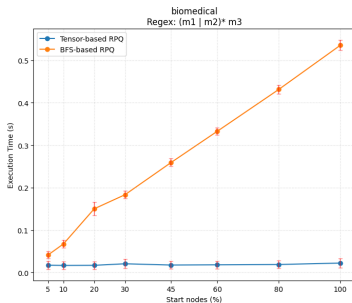
Performance of ms_bfs_based_rpq (Start Vertices: 25%)



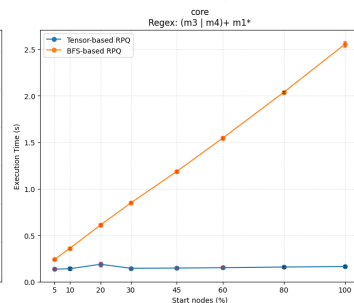
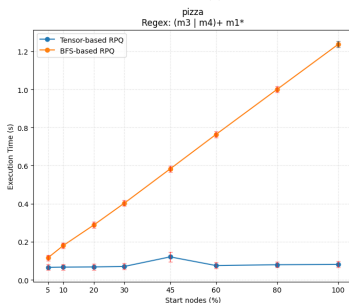
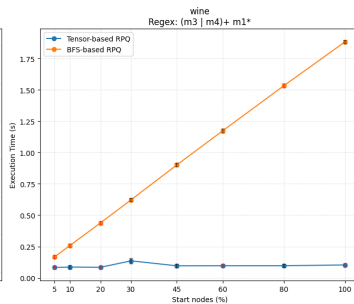
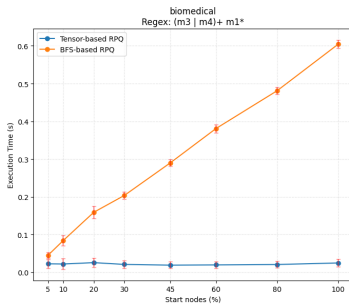
Вывод для ms_bfs_based_rpq

- Сравнение производительности:
 - ▶ **CSC** – наиболее быстрый формат
 - ▶ **CSR** – немного медленнее **CSC**, но часто сопоставим по времени
 - ▶ **LIL** – значительно медленнее
 - ▶ **DOK** – также медленнее, чем **CSR/CSC**
- Обоснование:
 - ▶ Арифметических операции
 - ▶ Транспонирование **CSC** даёт **CSR** (Транспонирование **CSR** даёт **CSC**), поэтому центральная формула этого алгоритма – **CSR @ front @ CSC** (при исходном **CSC**) – является более сбалансированной по определению перемножения матриц.

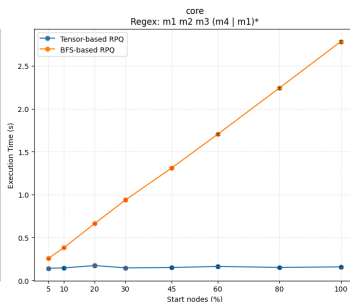
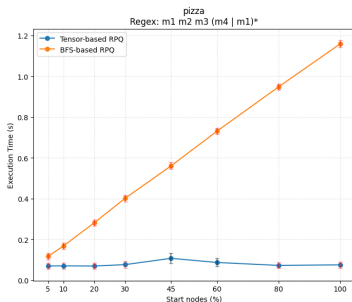
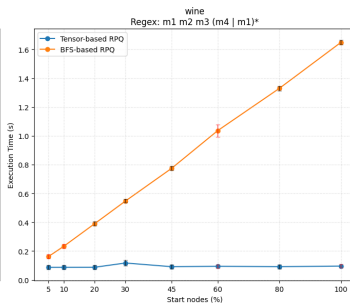
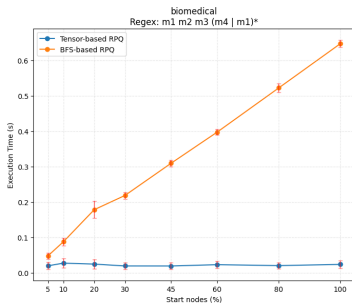
Результаты по вопросу №2 (1/4)



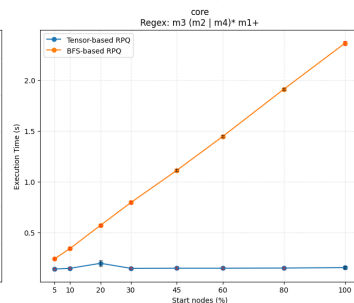
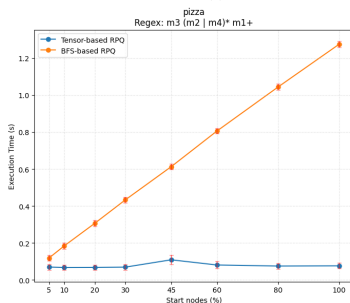
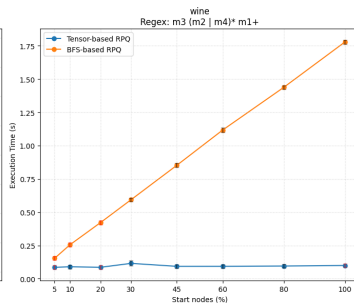
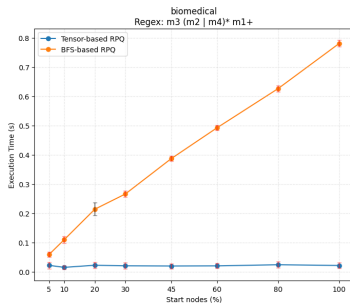
Результаты по вопросу №2 (2/4)



Результаты по вопросу №2 (3/4)



Результаты по вопросу №2 (4/4)



Вывод по вопросу №2

- Для выбранных графов и регулярных выражений нет размера стартового множества, при котором **ms_bfs_based_rpq** выгоднее **tensor_based_rpq**
- Обоснование:
 - ▶ **tensor_based_rpq** выполняет одно транзитивное замыкание через последовательные умножения разреженных матриц – операции, высокооптимизированные в `scipy.sparse`
 - ▶ **ms_bfs_based_rpq** использует множество итераций в Python-коде, в каждой из которых выполняется несколько матричных умножений, сравнений и созданий промежуточных матриц

Ссылка на pull request:

<https://github.com/RodionovMaxim05/formal-lang-course/pull/5>